

Arquivos

Algoritmos e Programação 2

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

Introdução

- Estruturas de dados manipuladas fora do ambiente do programa são conhecidas como **arquivos**
- Ambiente do programa é a **memória principal**
 - Nem sempre é possível armazenar
 - Nem sempre conveniente manter certas estruturas de dados

Arquivos em C

- Na linguagem C, **arquivos** representam diversas coisas: arquivos em disco, impressora, teclado ou qualquer dispositivo de entrada e saída
- Esta aula considera apenas **arquivos em disco**!!

Biblioteca stdio.h

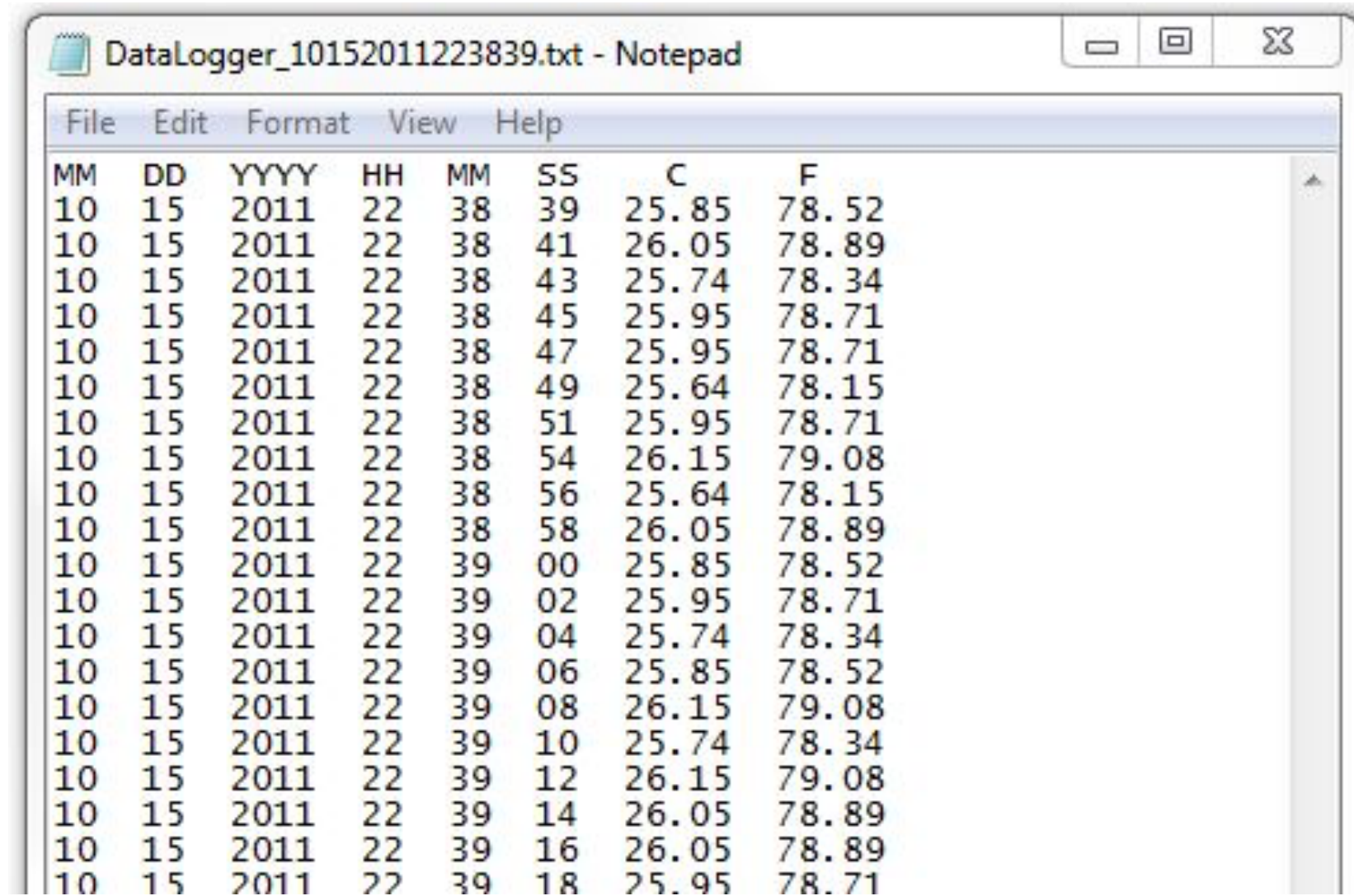
- C dá suporte à utilização de arquivos por meio da biblioteca **stdio.h**
- Essa biblioteca fornece funções para manipulação de arquivos, define novos tipos de dados, como o tipo **FILE**
 - Uma variável do tipo **FILE** é capaz de identificar um arquivo no disco, direcionando para ele todas as operações
 - A seguir, que podemos ver as duas variáveis, **arq** e **pont**, que podem referenciar arquivos em disco:

```
FILE *arq, *pont;
```

Tipos de Arquivos em C

- Em C, os dados podem ser gravados em arquivos binários ou de texto:
 - **Arquivos de texto:** podem ser lidos diretamente por qualquer editor de texto
 - **Arquivos binário:** devem ser lidos por programas especiais que convertem a cadeia de bits em informação compreensível

Arquivo Texto no Notepad



MM	DD	YYYY	HH	MM	SS	C	F
10	15	2011	22	38	39	25.85	78.52
10	15	2011	22	38	41	26.05	78.89
10	15	2011	22	38	43	25.74	78.34
10	15	2011	22	38	45	25.95	78.71
10	15	2011	22	38	47	25.95	78.71
10	15	2011	22	38	49	25.64	78.15
10	15	2011	22	38	51	25.95	78.71
10	15	2011	22	38	54	26.15	79.08
10	15	2011	22	38	56	25.64	78.15
10	15	2011	22	38	58	26.05	78.89
10	15	2011	22	39	00	25.85	78.52
10	15	2011	22	39	02	25.95	78.71
10	15	2011	22	39	04	25.74	78.34
10	15	2011	22	39	06	25.85	78.52
10	15	2011	22	39	08	26.15	79.08
10	15	2011	22	39	10	25.74	78.34
10	15	2011	22	39	12	26.15	79.08
10	15	2011	22	39	14	26.05	78.89
10	15	2011	22	39	16	26.05	78.89
10	15	2011	22	39	18	25.95	78.71

Cada byte representa um caractere (no caso de arquivo no formato ASCII).

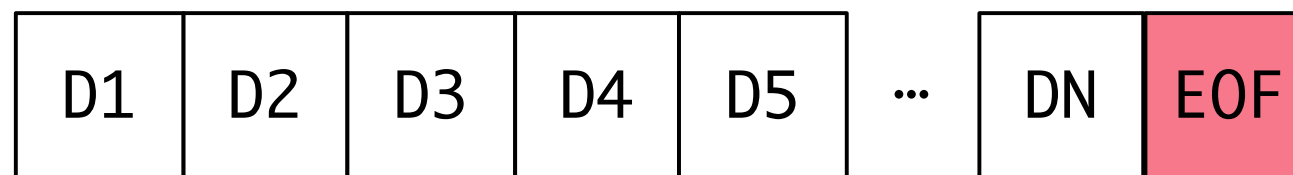
Arquivo Binário no Notepad

ÿØÿáExifl*ÿiDucky<ÿáhttp://ns.adobe.com/xap/1.0/ ÿiAdobedÄÿÜ ÿÄ±±ÿÄ@ !1AQaq"2i±BF
!G8*[J78ÒcmòÐdò%MiZOðálo6@'ó\$ä"Ólyhk"s!\$r t@9ñ8Í³BÖ«#nãääÄØZçÖùA3L°c&. 8:NDÖ
áÄHygB°Tê"IM°¥Ga# {P+CÄ»äliÄWO@bNgÉääÖÖÆIZÖI#ÿ1dY"éiÆ@+¬1Tê«piÄ±i" é¬Js
0BRa¶Y/A_Mý2{8ä}2-zpó'Up~nīmīYfrC#ō Éæ7Z+ógL+Ö¬33(ñ|DZHóf£#AÄN+ àFÆ" Jp ôj; 3
³cP°q%"HdÜiPÖ@Äð2#ÖXyøIÿY*ÉÚknßLKlājÖ"ô|~>è½kéÇ"CV¿>8_@çÿl_öÿjØ6~m\$NQLpli¿
Ty-é9«h,ä@äÄöeÜ ÍúÉwP¶ÿÿÜ¶[Í¼¤DQ±Ç)j³Pñ]*önòÄþéZSÜñÄý½NncÑu=ðç¬»Ó¬r({xaÖ@ëZI"
";k+W¶wiE½ÖbdjóÄjāñVÖú?_m_Y³¼U4äØÄ+ékgðV\ÜZÉ#4¿i·Tô&q_É×ÄV#?1äçtkàsÑ^9ää@
ñb&`@FJ±g\$æU¥i0ääÜz'D.b {±Çj²b¶¶T9ß`HHs9ÉêÆL°, %hsíÄ:Çéseí@ÉÄK1@§0F0½\$ijj¥4Ö|
Üð;#A'MIÄ&'CP«ç¥A@x×Ç 2:-Üé&¥|p:vçl5\$ý, ^Ä\$ðl« ðyâbðha+¼ÄHÆ.Ç½b'<,wçlKçÆ|q¶HVt
Gpø½±ðän°F *N@pà¾fz4¼"J§[ñÉ)äÄÄèðlþjÄÉüä{Í%mqÄ"É«ÿ :=;m%(%U@kZ«)S\k }hàNÜ
_*@äi"}Um\$Mg¾Ä"Üú(¥Z2ä_áÜsÖ×ÿÿik9¾¶Öayl-YH+æâ2áCù+NFeB0ðl 1-<ÄÇ 0*h)3lÿxEv
©Ö)ÉÜÐ+S@scj:@.½üþ3a à9á²-Z'ÍE"qSE@aT·ð É"Vçª) Qæ9X2øKxdÐ¥x;|pA*9oVrLi@i©Ä<
Sov°z|ÄRµâ|0Q°ÖÄsÄ@ikjâ"Ä*7Ö9ÄR_VaÖi ?Üh³1ÜH¾z>f g^ià nµ¾Ëµj^ÜñÜ«9^,rÆð×&¼il
lÍÄ¶jò2k3èø°ßc+)jü=ð½µ,0 AÜ9mªÉÉx_ zwÜÓsis6µTáð0Ug+"Yu-ó)Äé[bè,M#V\$S\$!8öpÄ½X
B·3lXlÉÜ|Qlp üÿ«½à(l°Ö u p²xäirUNð5ç+Lo¶ÆêWq Y)KpCgè35-M'ErJUÉc É'ÄÖÖmKZ\$mJ£:
+`R`mýéÖLèÄfñ©Üê«àLó_ ^mPönÜâ ëjÿl/ijñGjÄK|/Mu£Ä»ð°Ü@lÉ'Ü/lop¼j;f%·Ld)vÄ-Ü[C >¶Ö
jÉðÄÆÖÖfÜ×H\Ü»Ð cZ+`[Pk!úç§÷Yn»O§Ü!äñðÖCùÄUðBÉ+éae@¼h°ÿÓÿC-ÿÿ§'lç_ÉÜðQpDN
ùO½ÿ06)|Ði6+r",¿U(Pls=ßzPä§ëÿ_§_ ^ÿm%YäñÆóÆVÝPRµ8é~UN¶@SMÝ¼ØÜtäµzª/- ^8\
ð4ç|ÜÄ³L¼PAIäc'DVNtçðèl>Ü°Krä"Uìè>ÓbCðA³¼3Qä9eÜi*·èCnþ@ù¾P258¥5¶ÄçyeÖh\$Z?è
Öl3Élp<Ö3w¶l~<«§±Ö¥LKÝl_ :Äx66lél6ði<@µ·Xü* &foVç8UÖS¥;±è¥ÖæiRmèðnVFJ»l+5f zÉÜ
ÄÄçYØQ@®EpUwëb_É¬ÖÄØuQè¶© Ngú°D?Éæÿ_ÿÓvè2U«ÜQòsUä)Fil4NüØi«ÉÆ)TE³VB
JApJCéj-ú{ +l±zÄ@_ñÆêLÉT5\$ÖPr 2,6U|QÆl>8lP7°t4×0zvÖ l-@µl xWUÖäØuñ¾āmýln-x£:
ëjÖ-Ä,A|DÖ¬r5ÉjÄQKY%í½@µÓquÜ2²Ä±GD?6t@xlià%öywYÄ{ð»eÄ²Ü'pT ©(\$H:¥qÇèó+£N¿
l(>oß¥¬Äl3<@ççik²lðEná¼{ ØHd1_"_ü± ©AÜ<F^PnYé'ÜlÖÖèe>»ç0ødµçL|Çp³é|tl{§Ö/g·ÿë¶
/Éøc½¬³@ Ço@LÉ+ÉñÄ%6^t+Tny=m6lîläUMQäñ·äg·ÄJlµrSµýóÜ§-zçÖ8Ö[r""h²ª~ð+ÿ_ìvè
_lrÉ*x`É8ç@ÄqÄ¹»BälpF Wð§ÜU\$Eit_É'a^Ä7Bjtpä-)lÄ¶cQ ¥g²_Ü2Ö;ÜL±lµ?"æj3ä{üÐi²"ZØeA
çÄZwä×¥ÖÖ}ñ¾jA1m|PK F?7_änaYtEÿ÷"m,mïöRi<²GÆVéWSTñÖ°4'pG-·g'F{ì¹ß>ço¹Ü'Éma§
Ü;4y*òJp»"ÿÜ½·wß:&Üxê'äðGxt*Gü#%çRa7h¬_eYÓsz\ZÉ7)'Ü°ÖÜwPjy"iüF%Z'ðègI¾èn¾lÜ
¥OV9lçjÈcL HØ.ÜlâÆi)|@lâQjç`=9æYWLyí×'[Ö;>äg°óí \$(çÉBl' z6lNÜS3(6ØvèX£¾iäÜa¶U«
{KSWÇ¿NZ/ivçÖ± u&ðÉ'l»Ü6«'l¶lù³5ükF§°ëüG»rÖXù0ððl+ü¥mlóc^x+ðSç<+iibÓ§¶l+BU%V
hëáÄT:çUrlÄ¶l#PÜ/(lÖöÜÿ^á>*pcb·rðDðm6;D§ H\$?&¬:mÉ1RÆ ¼Hl*F8+SYBC ¾jµs"8\$Çñ%
aRäÄW³¿ Ü@yo'Vb« U\Pr LH7Q§Ö"-R*jrøpÄ\$ÄID í¬6|Äç+c±ÄCAL3d;É§zÉZqÄ¾l\$K@_Ü³#l
+CWCj5çäWÖ¿\$iä±ÆèùcQêYfi8oY'/lil'p"2_jæpkèÜÖÜ;±fDüä³ir^°B»¶@æS ¿§+9.ÿ'FàUU@ð
D_JÉÑ9-RIP ÖW!èXCÉlÖ#lüh8)æjÿjÿv¶¶j£1lÖ¶#Ä@lÄ pG¶lÄjéány²#1i±°b æ9Ö+`ð1 lã?9Ép©Ç
ç<½Üíÿ òüëiNð¥¼wKG^Üä"py/:=K\$:éÜNl6½ù:k¥rPl×uð_pÜoißmjlíF5JáEjjZæ1Öiòri@l_päYN
ñ[Ö5a!_ÉµäBpi* «1VÝQm³i×YbKðð"fldÉ¬\$4Ö/+ßULÜ(zl¶¶)ÖRYTW×DVÈNV" lri5ÉÉÿà' Øu|GÖ
ÄÖ"æÄ :3u0f/ðDÉ7ðçFW?xðl¬@YÄHph%¿.F²5w.ÄC¾ðlµ¾¼+ÄlHlæwFlllÜX1äpY7ÄlÄ·ä²h

Exemplo de quando se tenta abrir um arquivo binário como texto.

O que é um arquivo?

- Um arquivo é uma sequência de bytes (D1, D2, D3, D4, D5, ... DN)



- Terminada por uma marca de fim de arquivo (**EOF**).

Abrir Arquivo: fopen()

A função **fopen()** abre um arquivo, retornando o ponteiro associado a esse arquivo

```
FILE *p;  
p = fopen("nome_do_arquivo", "modo_de_abertura");
```

onde:

- **p:** armazenará o endereço inicial de memória ocupado por um arquivo (se, por qualquer motivo, o arquivo não puder ser aberto, a variável p receberá o valor **NULL**)
- **nome_do_arquivo:** é o nome do arquivo que se deseja abrir. Pode conter o caminho (*path*) da pesquisa
- **modo_de_abertura:** representa o modo como arquivo o será aberto

Modos Abertura Arquivo

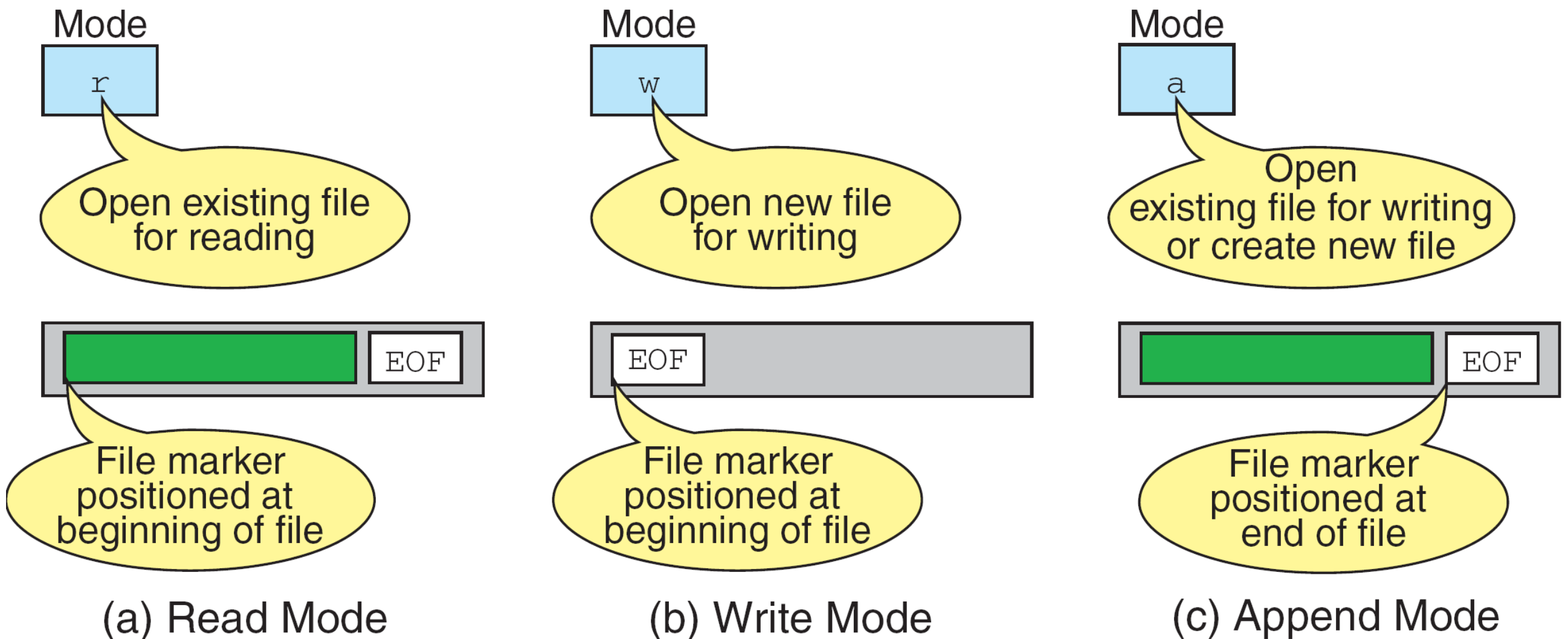
Modo	Descrição
r	Abre um arquivo de texto onde poderão ser realizadas apenas leituras.
w	Cria um arquivo de texto onde poderão ser realizadas apenas operações de escrita.
a	Anexa novos dados a um arquivo de texto.
rb	Abre um arquivo binário onde poderão ser realizadas apenas leituras.
wb	Cria um arquivo binário onde poderão ser realizadas apenas operações de escrita.
ab	Anexa novos dados a um arquivo binário.
r+	Abre um arquivo de texto onde poderão ser realizadas operações de leitura e de escrita.
w+	Cria um arquivo de texto onde poderão ser realizadas operações de leitura e de escrita.
a+	Anexa novos dados ou cria um arquivo de texto para operações de leitura e escrita.
rb+	Abre um arquivo binário onde poderão ser realizadas operações de leitura e de escrita.
wb+	Cria um arquivo binário onde poderão ser realizadas operações de leitura e de escrita.
ab+	Anexa novos dados a um arquivo binário para operações de leitura e de escrita.

Modos Abertura Arquivo (Cont.)

Modo	Descrição
r	Abre arquivo de texto no modo leitura: • Se o arquivo existir, marcador do arquivo é posicionado no início. • Se o arquivo não existir, retorna-se erro.
w	Abre arquivo no modo escrita: • Se o arquivo existir, este é apagado. • Se o arquivo não existir, é criado.
a	Abre arquivo de texto no modo anexar: • Se o arquivo existir, o marcador do arquivo será posicionado no final. • Se o arquivo não existir, ele será criado.

Comportamento dos modos de abertura quando um arquivo de mesmo nome existir e não existir.

Mais em Modos de Abertura



Exemplo 1: Crie um Arquivo

```
1. #include <stdio.h>
2.
3. int main() {
4.     // declare um ponteiro para arquivo
5.     FILE *arq;
6.
7.     // abra um arquivo no modo escrita
8.     arq = fopen("arquivo.txt", "w");
9.
10.    return 0;
11.}
```


Exemplo 2: Verifique Erro Após Abrir

```
1. #include <stdio.h>
2.
3. int main () {
4.     FILE *arq;
5.     arq = fopen("arquivo.txt", "r");
6.
7.     // Verifique se arquivo foi aberto corretamente
8.     if(arq == NULL) {
9.         // Provavelmente arquivo nao existe
10.        perror("Erro ao abrir o arquivo");
11.    }
12.    else {
13.        /* As demais instruções do programa só poderão ser executadas
14.        se o arquivo foi aberto corretamente */
15.        fclose(arq);
16.    }
17.
18.    return 0;
19.}
```

```
%
Erro ao abrir o arquivo: No such file or directory
```

Arquivos Texto

Ler e Escrever em Arquivo Texto

- **fprintf** é similar ao **printf**, porém com a diferença que escrevemos no arquivo ao invés da tela
- **fscanf** é similar ao **scanf**, porém com a diferença que lemos do arquivo ao invés do teclado

Exemplo 3: Gere 100 Aleatórios

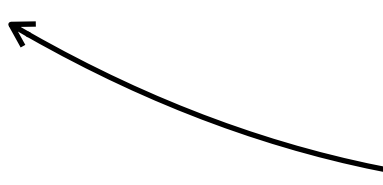
Faça um programa para gerar 100 números aleatórios no intervalo de 1-100 e escreva estes números no arquivo *100_aleatorios.txt*.

Exemplo 3: Solução

```
1. #include <stdio.h>
2. #include <time.h>
3. #include <stdlib.h>
4.
5. int main() {
6.     int i, r;
7.     FILE *f;
8.
9.     // abra arquivo
10.    f = fopen("100_aleatorios.txt", "w");
11.    if (f == NULL) {
12.        perror("Erro ao abrir o arquivo");
13.        exit(1);
14.    }
15.
16.    // inicialize o gerador de numeros aleatorios
17.    // com a hora atual como semente
18.    srand(time(NULL));
19.    for (i = 0; i < 100; i++) {
20.        //
```

```
1. #include <stdio.h>
2. #include <time.h>
3. #include <stdlib.h>
4.
5. int main() {
6.     int i, r;
7.     FILE *f;
8.
9.     // abra arquivo
10.    f = fopen("100_aleatorios.txt", "w");
11.    if (f == NULL) {
12.        perror("Erro ao abrir o arquivo");
13.        exit(1);
14.    }
15.
16.    // inicialize o gerador de numeros aleatorios
17.    // com a hora atual como semente
18.    srand(time(NULL));
19.    for (i = 0; i < 100; i++) {
20.        // gere um numero aleatorio no intervalo 1 a 100
21.        r = (rand() % 100) + 1;
22.
23.        // escreva o numero no arquivo
24.        fprintf(f, "%d ", r);
25.    }
26.    fclose(f);
27.    return 0;
28.}
```

printf envia a saída para o dispositivo padrão de saída (geralmente o console), enquanto fprintf envia para um arquivo ou outro fluxo especificado pelo usuário.



Exemplo 4: Encontre o Maior

Leia o arquivo *100_aleatorios.txt*, e encontre o maior número gerado. Imprima na tela esse número.

Exemplo 4: Solução

```
1. #include <stdio.h>
2. #include <stdlib.h>
3.
4. int main() {
5.     int i, maior, r;
6.     FILE* f;
7.
8.     f = fopen("100_aleatorios.txt", "r");
9.     if (f == NULL) {
10.         perror("Erro ao abrir o arquivo");
11.         exit(1);
12.     }
13.     maior = 0;
14.     for (i = 0; i < 100; i++) {
15.         // leia um inteiro do arquivo
16.         fscanf(f, "%d", &r);
17.
18.         // se inteiro lido foi maior que o maior inteiro ate entao
19.         if (r > maior)
20.             maior = r;
21.     }
22.     printf("%d\n", maior);
23.     fclose(f);
24.     return 0;
25. }
```

scanf lê da entrada padrão (normalmente o teclado), enquanto fscanf lê de um arquivo especificado.



Exemplo 5: Leia Até Fim do Arquivo

Imagine agora o seguinte cenário: você deve abrir e ler um arquivo que possui números inteiros espaçados e imprimir na tela todos os números. Porém, você não sabe a priori quantos números existem salvos no arquivo.

Função feof

feof

<stdio>

```
int feof ( FILE * stream );
```

Check end-of-file indicator

Checks whether the **end-of-File indicator** associated with **stream** is set, returning a value different from zero if it is.

This indicator is generally set by a previous operation on the **stream** that attempted to read at or past the end-of-file.

Notice that **stream**'s internal position indicator may point to the **end-of-file** for the next operation, but still, the **end-of-file** indicator may not be set until an operation attempts to read at that point.

This indicator is cleared by a call to [clearerr](#), [rewind](#), [fseek](#), [fsetpos](#) or [freopen](#). Although if the **position indicator** is not repositioned by such a call, the next i/o operation is likely to set the indicator again.

Parameters

stream

Pointer to a [FILE](#) object that identifies the stream.

Return Value

A non-zero value is returned in the case that the **end-of-file indicator** associated with the stream is set. Otherwise, zero is returned.

Link: <https://cplusplus.com/reference/cstdio/feof/?kw=feof>.

Exemplo 5: Solução feof (não usem!!)

```
1. #include <stdio.h>
2. #include <stdlib.h>
3.
4. int main() {
5.     int r, nread;
6.     FILE *f;
7.
8.     // abra o arquivo
9.     f = fopen("entrada.txt", "r");
10.    if (f == NULL) {
11.        printf("Erro ao abrir o arquivo!\n");
12.        exit(1);
13.    }
14.    // enquanto nao chegou no final do arquivo
15.    while (!feof(f)) {
16.        // leia um inteiro do arquivo
17.        fscanf(f, "%d ", &r);
18.        printf("%d ", r);
19.    }
20.    fclose(f);
21.    return 0;
22.}
```

Atenção

Existe uma diferença que pode ocorrer no Windows, Linux ou Mac dependendo do compilador utilizado.

Em alguns compiladores basta o **ponteiro** estar no fim do arquivo para ativar o **feof**. Em outros casos é requerido que além do **ponteiro** estar no final, tente-se ler um dado (**fscanf**) para ativar o **feof**.

Exemplo 5: Solução Recomendada

```
1. #include <stdio.h>
2. #include <stdlib.h>
3.
4. int main() {
5.     int r, n;
6.     FILE *f;
7.
8.     // abra o arquivo
9.     f = fopen("entrada.txt", "r");
10.    if (f == NULL) {
11.        perror("Erro ao abrir o arquivo!\n");
12.        exit(1);
13.    }
14.    while (1) {
15.        // leia um inteiro do arquivo
16.        n = fscanf(f, "%d ", &r);
17.
```

```
1. #include <stdio.h>
2. #include <stdlib.h>
3.
4. int main() {
5.     int r, n;
6.     FILE *f;
7.
8.     // abra o arquivo
9.     f = fopen("entrada.txt", "r");
10.    if (f == NULL) {
11.        perror("Erro ao abrir o arquivo!\n");
12.        exit(1);
13.    }
14.    while (1) {
15.        // leia um inteiro do arquivo
16.        n = fscanf(f, "%d ", &r);
17.
18.        // se chegou ao final do arquivo, finalize
19.        if (n == EOF)
20.            break;
21.
22.        printf("%d ", r);
23.    }
24.    fclose(f);
25.    return 0;
26.}
```

Quando nenhuma leitura é feita, fscanf retorna a constante EOF (-1)



Referências

- ASCENCIO, A. F. G.; CAMPOS, E. A. V. Fundamentos da programação de computadores: algoritmos, Pascal, C/C++ e Java. 3. ed. São Paulo: Pearson, 2012.
- DEITEL, P.; DEITEL, H. C: Como Programar. 6ª ed. São Paulo: Pearson, 2011.
- PIVA, D.J. et al. Algoritmos e programação de computadores. Rio de Janeiro: Elsevier, 2012.